Weighbridge Weight Capture System

Implementation Guide: Part 1 (Indicator to Frontend) and Integration with Part 2

TruckMate Weighbridge Management System

14 October 2026

Contents

1 Overview **3**

2 System Architecture **3**

3 Step 0: Learn the Indicator Protocol **3**

4 Step 1: Edge Agent (.exe) **4**

4.1 Create the project 4

4.2 Configuration 4

4.3 Core logic (Worker.cs) 4

4.4 Agent robustness 5

4.5 Build the executable 6

4.6 Install as a Windows Service 6

5 Step 2: Server Weight API **6**

5.1 In-memory latest-weight cache 6

5.2 API controller 6

6 Step 3: Frontend Live Weight Display **7**

7 Step 4: Connecting Part 1 to Part 2 **7**

7.1 Implement IScaleService with real data 7

7.2 Swap the registration 8

7.3 Capture the weight on the server 8

7.4 Multiple weighbridges (optional) 8

8 Suggested Build Order **8**

9 Common Pitfalls and Production Notes **9**

1 Overview

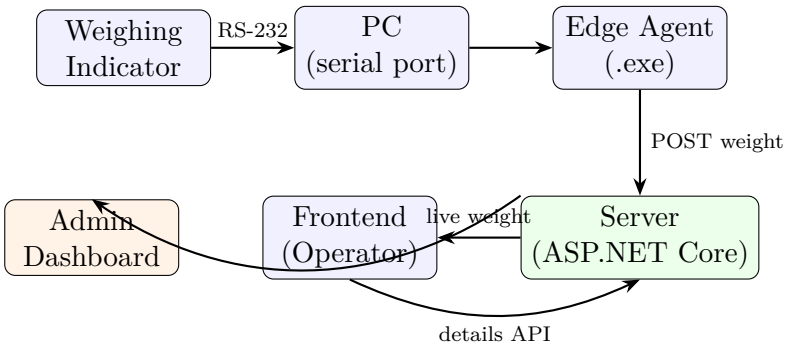
The project has two parts:

1. **Part 1 – Weight capture.** Read the weight from the weighing indicator, pass it through a PC edge agent to the server, and show it live in the frontend.
2. **Part 2 – Operator details.** The operator enters transaction details (vehicle, driver, material, and so on) in the frontend. These are passed to the server, and the admin views them in the admin dashboard. This part is already implemented in the TruckMate repository (ASP.NET Core MVC, .NET 10, Entity Framework Core, SQLite).

The existing application already contains an `IScaleService` abstraction with a `MockScaleService`. Part 1 therefore consists of building the components that feed real weight data into that abstraction.

Key design rule. The server is the source of truth for weight. The browser only *displays* the live weight. When the operator saves a weighing, the server reads the latest weight from its own cache; the browser never submits the weight value. This prevents tampering.

2 System Architecture



Component	Responsibility	Technology
Indicator	Measures and outputs weight over serial	Hardware
Edge Agent	Reads COM port, parses, detects stability, posts to server	.NET Worker Service
Weight API	Receives weight, caches latest, serves frontend	ASP.NET Core
Frontend	Shows live weight, enables Save only when stable	Razor + JavaScript
Details API	Stores transaction details (Part 2)	Existing controllers
Admin Dashboard	Views all transactions (Part 2)	Existing views

3 Step 0: Learn the Indicator Protocol

Complete this step before writing any code, because every later step depends on it.

1. Obtain the indicator manual and find the serial settings (baud rate, data bits, parity, stop bits; commonly 9600/8/N/1) and the output mode (continuous or on-demand).

2. Connect the indicator to the PC with RS-232 or a USB-serial adapter and open it in a serial terminal such as PuTTY, Tera Term, or RealTerm.
3. Record several raw lines. They typically resemble `ST,GS,+001234.5kg` or `+ 1234 kg`, ending in a carriage return and line feed. Note which part indicates stable versus unstable (often `ST`/`US`), the units, and the sign.

4 Step 1: Edge Agent (.exe)

The edge agent is a small .NET Worker Service running on the weighbridge PC. It reads the COM port, parses each line, and posts the result to the server.

4.1 Create the project

```
dotnet new worker -n WeighAgent
cd WeighAgent
dotnet add package System.IO.Ports
dotnet add package Microsoft.Extensions.Hosting.WindowsServices
```

4.2 Configuration

Keep site-specific values in `appsettings.json` so they can change without recompiling.

```
{
  "Agent": {
    "ScaleId": "WB1",
    "ComPort": "COM1",
    "BaudRate": 9600,
    "ServerUrl": "https://your-server/api/weight",
    "ApiKey": "long-random-secret",
    "StableCount": 5,
    "ToleranceKg": 0
  }
}
```

4.3 Core logic (Worker.cs)

```
public class Worker : BackgroundService
{
    private readonly IConfiguration _cfg;
    private readonly IHttpClientFactory _http;
    private readonly ILogger<Worker> _log;
    private double? _last; private int _same;

    public Worker(IConfiguration cfg, IHttpClientFactory http, ILogger<Worker> log)
    { _cfg = cfg; _http = http; _log = log; }

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            try { await RunAsync(ct); }
```

```
            catch (Exception ex)
            {
                _log.LogError(ex, "Serial loop failed, retrying in 5s");
                await Task.Delay(5000, ct);
            }
        }
    }

    private async Task RunAsync(CancellationToken ct)
    {
        var a = _cfg.GetSection("Agent");
        using var port = new SerialPort(a["ComPort"], int.Parse(a["BaudRate"]!),
            Parity.None, 8, StopBits.One)
        { NewLine = "\r\n", ReadTimeout = 3000 };
        port.Open();

        var client = _http.CreateClient();
        client.DefaultRequestHeaders.Add("X-Api-Key", a["ApiKey"]);

        while (!ct.IsCancellationRequested)
        {
            string line;
            try { line = port.ReadLine(); }
            catch (TimeoutException) { continue; }

            if (!TryParse(line, out var kg)) continue;

            // stability: same value N times in a row
            _same = (_last == kg) ? _same + 1 : 1;
            _last = kg;
            bool stable = _same >= int.Parse(a["StableCount"]!);

            await client.PostAsJsonAsync(a["ServerUrl"], new {
                scaleId = a["ScaleId"], weightKg = kg, stable,
                timestamp = DateTime.UtcNow
            }, ct);
        }
    }

    // ADAPT this regex to the indicator's real output
    static readonly Regex Rx = new(@"([+-]?\s*\d+(\.\d+)?)\s*kg",
        RegexOptions.IgnoreCase);

    static bool TryParse(string line, out double kg)
    {
        kg = 0;
        var m = Rx.Match(line);
        return m.Success && double.TryParse(m.Groups[1].Value.Replace(" ", ""),
            NumberStyles.Float, CultureInfo.InvariantCulture, out kg);
    }
}
```

4.4 Agent robustness

- **Reconnect on failure.** The outer loop retries after unplugged cables or a temporarily unreachable server.
- **Do not flood the server.** Indicators often send ten or more lines per second. Post at most about four times per second, or only when the value changes plus a one-second heartbeat.

- **Log to a file.** Log files are essential for on-site debugging.

4.5 Build the executable

```
dotnet publish -c Release -r win-x64 --self-contained true -p:PublishSingleFile=true
```

4.6 Install as a Windows Service

Add `builder.Services.AddWindowsService();` in the agent's `Program.cs`, then run the following in an administrator PowerShell so the agent starts automatically on boot:

```
sc.exe create WeighAgent binPath= "C:\WeighAgent\WeighAgent.exe" start= auto
sc.exe start WeighAgent
```

5 Step 2: Server Weight API

5.1 In-memory latest-weight cache

```
public record WeightReading(string ScaleId, double WeightKg,
                            bool Stable, DateTime Timestamp);

public class WeightCache
{
    private readonly ConcurrentDictionary<string, (WeightReading R, DateTime ReceivedUtc)> _d
    = new();

    public void Update(WeightReading r) => _d[r.ScaleId] = (r, DateTime.UtcNow);

    public (WeightReading R, DateTime ReceivedUtc)? Get(string scaleId)
        => _d.TryGetValue(scaleId, out var v) ? v : null;
}
```

Register it in `Program.cs`:

```
builder.Services.AddSingleton<WeightCache>();
```

5.2 API controller

```
[ApiController, Route("api/weight")]
public class WeightApiController : ControllerBase
{
    private readonly WeightCache _cache;
    private readonly IConfiguration _cfg;
    public WeightApiController(WeightCache c, IConfiguration cfg)
    { _cache = c; _cfg = cfg; }

    // Called by the Edge Agent. Uses an API key instead of cookie login.
    [AllowAnonymous, HttpPost]
    public IActionResult Post([FromBody] WeightReading r)
    {
```

```
        if (Request.Headers["X-Api-Key"] != _cfg["Agent:ApiKey"])
            return Unauthorized();
        _cache.Update(r);
        return Ok();
    }

    // Called by the frontend. Requires a logged-in operator.
    [Authorize, HttpGet("latest")]
    public IActionResult Latest(string scaleId = "WB1")
    {
        var v = _cache.Get(scaleId);
        if (v is null) return Ok(new { online = false });
        var age = (DateTime.UtcNow - v.Value.ReceivedUtc).TotalSeconds;
        return Ok(new { online = age < 5,
                        weightKg = v.Value.R.WeightKg,
                        stable = v.Value.R.Stable });
    }
}
```

Store the agent API key in `appsettings.json` for development and in an environment variable in production. Because TruckMate uses cookie authentication, the `[AllowAnonymous]` attribute combined with the API key check is what prevents the agent from being redirected to the login page.

6 Step 3: Frontend Live Weight Display

Start with simple polling, which is easy and reliable. SignalR can be adopted later if push updates are required.

```
<div id="weight" class="display-3">-- kg</div>
<span id="status"></span>
<script>
setInterval(async () => {
  const r = await fetch('/api/weight/latest?scaleId=WB1');
  const d = await r.json();
  document.getElementById('weight').textContent =
      d.online ? d.weightKg.toFixed(1) + ' kg' : '-- kg';
  document.getElementById('status').textContent =
      !d.online ? 'Scale offline' : d.stable ? 'Stable' : 'Unstable';
  document.getElementById('saveBtn').disabled = !(d.online && d.stable);
}, 500);
</script>
```

Disabling the Save button until the weight is stable is a useful operator-safety feature.

7 Step 4: Connecting Part 1 to Part 2

7.1 Implement IScaleService with real data

Create an implementation that reads from the cache instead of the mock. Adapt the method and type names to the actual `IScaleService` interface in the repository.

```
public class AgentScaleService : IScaleService
{
    private readonly WeightCache _cache;
```

```
    public AgentScaleService(WeightCache c) => _cache = c;

    public Task<ScaleSnapshot> GetSnapshotAsync(string scaleId = "WB1")
    {
        var v = _cache.Get(scaleId);
        if (v is null ||
            (DateTime.UtcNow - v.Value.ReceivedUtc).TotalSeconds > 5)
            throw new InvalidOperationException("Scale offline or data stale");
        return Task.FromResult(
            new ScaleSnapshot(v.Value.R.WeightKg, v.Value.R.Stable));
    }
}
```

7.2 Swap the registration

Use a configuration flag so the mock can still be used during development:

```
if (builder.Configuration.GetValue<bool>("Scale:UseMock"))
    builder.Services.AddScoped<IScaleService, MockScaleService>();
else
    builder.Services.AddScoped<IScaleService, AgentScaleService>();
```

7.3 Capture the weight on the server

In the existing weighing controller action, replace any weight value coming from the form with the server-side snapshot:

```
var snap = await _scale.GetSnapshotAsync("WB1");
if (!snap.Stable) return BadRequest("Weight not stable");
transaction.FirstWeight = snap.WeightKg;   // or SecondWeight
transaction.FirstWeightTime = DateTime.Now;
```

The operator detail forms, pending and completed states, ticket PDF generation, and the admin dashboard need no changes, because they simply read the saved weights from the database.

7.4 Multiple weighbridges (optional)

The `ScaleId` field already supports multiple scales. Add a `WeighbridgeId` column to the transaction and have each agent post under its own identifier.

8 Suggested Build Order

1. Use a serial terminal to confirm the raw indicator data (Step 0).
2. Build the agent as a console app that only prints parsed weights.
3. Add the server endpoint and cache, and test it with Postman or curl before using the real agent:

```
curl -X POST https://localhost:7xxx/api/weight \
  -H "X-Api-Key: secret" -H "Content-Type: application/json" \
```

```
-d "{\"scaleId\":\"WB1\",\"weightKg\":12340,\"stable\":true,\"timestamp\":\"2026-10-14
  T10:00:00Z\"}"
```

4. Make the agent post to the server.
5. Build the frontend live display.
6. Swap in `AgentScaleService` and make the save action use the server-side weight.
7. Publish the .exe, install it as a Windows Service on the weighbridge PC, and test end to end with a real truck.

9 Common Pitfalls and Production Notes

- **Do not test hardware blind.** Until the indicator is available, write a small fake serial sender or use a virtual COM port pair (for example com0com) so the agent can be tested on a development laptop.
- **Network.** The PC must be able to reach the server. Use HTTPS for a cloud-hosted server. For a LAN server, use a static IP or hostname and open the firewall port.
- **Stale data.** Always reject weights older than a few seconds, otherwise a weight recorded before a cable was unplugged could be saved.
- **Production hardening.** Replace the seeded development accounts, move away from SQLite for multi-user use, and keep the API key out of source control.